

SAM DUCKWORTH PHOTOGRAPHY

Online Shop — End-to-End Setup Plan

How to turn the existing “Framed Editions” page into a real shop that takes payment and fulfils orders — starting with simple **manual fulfilment**, wired into your current React + Supabase + Vercel site, with a clean path to full automation later.

Companion to *Framed Print Suppliers (AU) — Comparison*.

June 2026 · grounded in the current codebase (`src/App.tsx`, the `shop_collections_flags` migration, `source_path`).

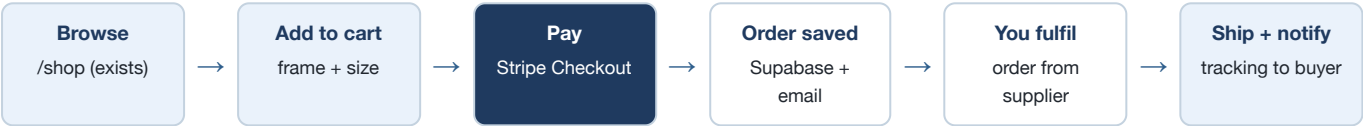
Recommendation at a glance

Use Stripe Checkout + a small Supabase `orders` table + two Vercel serverless functions, and fulfil manually via Prodigy (white-label dropship).

- It's the lowest-fee option (~1.7% + 30c, no monthly), it stays entirely inside your existing stack, and Stripe hosts the secure payment page so you handle no card data.
- You already have 80% of the storefront. The missing piece is checkout + order capture — ~1–2 days of dev for a working MVP.
- “Manual” means: customer pays online → you get an email + an order in `/admin` → you place the print order with your supplier (shipped straight to the customer) → you mark it shipped. No automation required to launch.
- The **same** setup automates later by adding one webhook that calls the supplier’s API — no re-platforming.

Fastest-to-launch alternative if you'd rather not write a backend: **Snipcart** (drop-in cart, ~2% fee) — see the options table. Both are good; Stripe is cheaper and more integrated, Snipcart is less code.

The journey, end to end



Where the shop is today

Good news: the storefront is already built and on-brand. From the codebase:

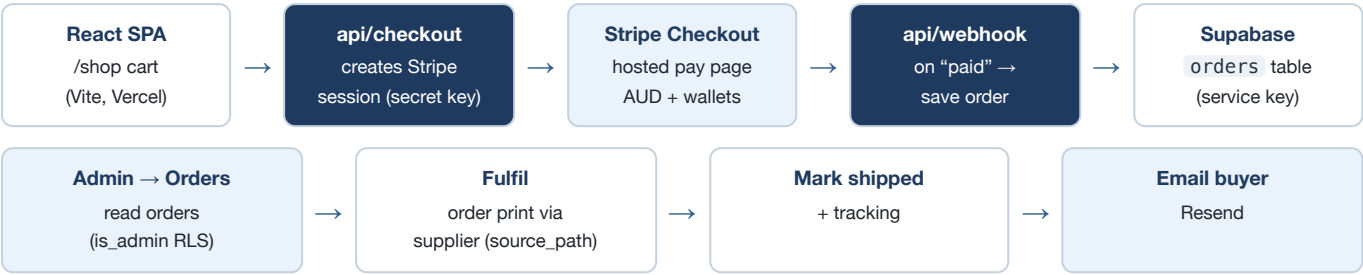
Already there	Detail (in the code)
<code>/shop</code> “Framed Editions” page	<code>ShopPage</code> in <code>src/App.tsx</code> — hero, “solid oak · archival matte · ready to hang · ships worldwide” strip, product grid, region filters.
Product curation	<code>photos.in_shop</code> / <code>shop_order</code> (admin picks which photos sell & their order). Banner photos + preview wall via <code>site_settings</code> .
Coming-soon gate	<code>site_settings.shop_public</code> (currently false). Public sees a “collection glimpse + coming soon”; admin sees the real shop. Toggle in Admin → Visibility .
Sizes & prices	<code>SHOP_SIZES</code> constant: A3 \$160 · A2 \$260 · A1 \$390, single oak frame. Size picker + price render per product.
Fulfilment hook	<code>photos.source_path</code> — admin-only absolute path to each original full-res file (“the find-the-original-to-sell-a-print link”). Exactly what you need to order a print.

What’s missing to actually sell

Gap	What’s needed
A real cart	Today “Add to cart” just increments a counter (<code>setCart(c=>c+1)</code>) — no line items, frame choice, or quantities.
Checkout & payment	No payment provider. Need Stripe (or similar) to take AUD, incl. Apple/Google Pay.
Order capture	No <code>orders</code> table. Need to record who bought what, where to ship, and fulfilment status.
Frame/glazing options	Only oak today. If you offer black/white or acrylic-vs-glass, the product + pricing model needs those variants.
Emails	Order confirmation to the buyer + new-order alert to you (and later, “shipped + tracking”).
Money admin	Stripe account, refund policy, GST/ABN handling, shipping pricing.

Recommended architecture

Keep everything in the stack you already run. The only new moving parts are **Stripe** and **two tiny Vercel serverless functions** (the `api/` folder Vercel auto-detects) — no new hosting, no re-platform.



Why this fits you: it mirrors patterns already in the repo — the webhook uses the **service-role key** server-side (same as your import scripts) to write orders bypassing RLS; the admin reads them through the authenticated role + `is_admin()` (same as photos). Stripe Checkout is **hosted**, so you’re never handling card numbers (PCI stays with Stripe).

One process note: the new `orders` table + `site_settings` needs a SQL migration. As always on this project, the service key can’t run DDL — you’ll paste the migration into the Supabase SQL editor by hand (the file goes in `supabase/migrations/`). Data after that flows through the service key as normal.

Options compared

Option	Cost	Dev effort	Pros	Cons	Verdict
Stripe Checkout + Supabase + Vercel	~1.7%+30c domestic (~3.5% intl). No monthly.	~1–2 days MVP	Lowest fees; fully in your stack; hosted/secure; AUD + Apple/Google Pay; scales to auto-dropship	You write a little backend (2 functions + table)	Recommended
Snipcart	2% of sales (min ~US\$20/mo) + gateway fee	~half a day	Drop-in cart via data-attributes; minimal code; handles cart/checkout UI	Higher fee; another vendor; still need Stripe behind it	Fastest launch
Shopify (Starter/Buy Button)	~A\$7–\$40/mo + ~1.7%+30c (or +extra if not Shopify Payments)	~half–1 day	Mature orders/inventory/email; buy-buttons embed; apps for dropship	Monthly fee; data/orders live in Shopify, not your Supabase; heavier	Good if you want it all off-the-shelf
Ecwid / Square Online	Free tier + payment fees	~half day	Free embeddable store; quick	Branding/limits on free tier; another silo	Budget off-the-shelf
Lemon Squeezy / Paddle (MoR)	~5% + fees (they handle global tax)	~half day	Merchant-of-record removes tax headaches	Built for digital goods; awkward for physical prints + shipping	Not ideal for framed prints

All the “off-the-shelf” options keep orders in their own dashboard rather than your Supabase, which is fine for manual fulfilment but means a second login and a weaker tie-in to your `source_path` workflow. The Stripe-native route keeps one source of truth.

Phased roadmap

Phase 0 · this week · no code ~½ day of decisions

Decide the product & supplier

- Pick the launch supplier from the comparison (recommended: **Prodigi** for white-label dropship; or Pixel Perfect / Lucent for a premium AU tier). Open the account, get the real A3/A2/A1 framed cost in each finish.
- Decide frame finishes to offer (e.g. **oak + black + white**) and **glazing = acrylic for posted items** (glass only for local pickup / premium).
- Set retail prices at ~2.5–3× landed cost. Re-check A1 (your \$390 may be tight — see the comparison's Pricing & Margin).
- Get an ABN if you don't have one; note GST only applies once turnover hits A\$75k (confirm with your accountant). Draft a simple shipping + returns/breakage policy.

Phase 1 · MVP checkout · manual fulfilment ~1–2 days dev

Make it actually sell

- Replace the counter cart with a real cart holding line items (`photoId, frame, size, qty, unitPrice`).
- Add `api/checkout` (creates a Stripe Checkout Session) and `api/stripe-webhook` (on `checkout.session.completed` writes the order to Supabase + emails you and the buyer).
- Create the `orders` migration (run by hand in Supabase SQL editor).
- Add frame-finish selector to `ShopProduct`; move `SHOP_SIZES/prices` to DB or env so you can edit without a deploy.
- Wire confirmation emails (Resend). Test with Stripe test mode, then flip `shop_public` → true to go live.

Outcome: customers can buy; you fulfil each order by hand. This is the “get moving” milestone.

Phase 2 · admin orders panel ~1 day dev

Manage orders without leaving your site

- Add an **Orders** tab to `/admin`: list with status (`new` → `paid` → `ordered` → `shipped` → `delivered`), customer + address, line items, and a one-click **packing slip** that shows each photo's `source_path` and chosen frame/size for fast supplier ordering.
- “Mark shipped” captures a tracking number and emails the buyer automatically.

Phase 3 · automate dropship ~1–2 days dev

Hands-off fulfilment

- Extend the webhook: on payment, call the supplier API (e.g. Prodigi) to create the print order automatically, shipped to the customer in your branding.
- Sync tracking back to the order + buyer email. You move from “place every order” to “glance at a dashboard”.

Data model addition

One new table. Orders are written **server-side by the webhook** (service key), and read by you (admin). Customers never read the table directly.

```
-- supabase/migrations/2026__orders.sql (run by hand in the SQL editor)
create table public.orders (
  id                uuid primary key default gen_random_uuid(),
  created_at        timestamptz not null default now(),
  stripe_session    text unique,           -- idempotency: one order per session
  status            text not null default 'paid', -- paid-ordered-shipped-delivered
  email             text not null,
  customer_name     text,
  ship_address      jsonb,                 -- from Stripe
  items             jsonb not null,        -- [{photo_id,title,frame,size,qty,price}]
  amount_total      int,                   -- cents
  currency          text default 'aud',    -- from Stripe
  supplier          text,                  -- e.g. 'prodigi'
  -- Sam Duckworth Photography · Online Shop — End-to-End Setup Plan · June 2026
```

```
supplier_order text,          -- filled in Phase 3
tracking       text,
notes         text
);
alter table public.orders enable row level security;
-- admins read/update; nobody else. Inserts come from the service key (bypasses RLS).
create policy "admins read orders" on public.orders for select to authenticated using (public.is_admin());
create policy "admins update orders" on public.orders for update to authenticated using (public.is_admin()) with check
(public.is_admin());
grant select, update on public.orders to authenticated;
```

Prices/sizes/frames can stay in code for v1, or move to a tiny `shop_products` / `site_settings` entry so you can change them without a deploy (recommended once you have >1 frame finish).

Concrete changes to the current code

File / area	Change
<code>src/App.tsx</code> — <code>ShopProduct</code>	Add a frame-finish selector (oak/black/white) + glazing note; on “Add to cart”, push a real line item (photo, finish, size, qty, price) into cart state instead of incrementing a counter.
<code>src/App.tsx</code> — <code>ShopPage</code>	Add a cart drawer/summary + “Checkout” button that POSTs the cart to <code>/api/checkout</code> and redirects to Stripe. Handle <code>/shop?success</code> / <code>?canceled</code> return states.
<code>SHOP_SIZES</code> constant	Expand to a price matrix by (finish × size), or load from DB/env. Keep A3/A2/A1; add finishes you offer.
new <code>api/checkout.ts</code>	Vercel serverless function: validates cart, looks up prices server-side (don’t trust client prices), creates a Stripe Checkout Session with shipping address collection, returns the URL.
new <code>api/stripe-webhook.ts</code>	Verifies Stripe signature; on <code>checkout.session.completed</code> inserts the order (service key) and triggers emails. Idempotent on <code>stripe_session</code> .
<code>src/lib/supabase.ts</code>	Add <code>getOrders()</code> / <code>updateOrderStatus()</code> (authenticated/admin) for the admin panel.
<code>/admin</code> dashboard	New “Orders” view (Phase 2): list, statuses, packing slip with <code>source_path</code> , “mark shipped” + tracking.
Env (Vercel + <code>.env.local</code>)	Add <code>STRIPE_SECRET_KEY</code> , <code>STRIPE_WEBHOOK_SECRET</code> , <code>RESEND_API_KEY</code> (server-only — not <code>VITE_</code> vars). Reuse the existing <code>SUPABASE_SERVICE_ROLE_KEY</code> in the webhook.
Go-live	Flip <code>site_settings.shop_public</code> → true (Admin → Visibility). Everything else already respects that flag.

Security must-do: always compute prices on the server in `api/checkout` from your own data — never charge the amount the browser sends, or a customer could edit it. Same fail-closed mindset as your `source_path` column lock.

Manual fulfilment runbook (Phase 1)

What happens for each sale, by hand, until you automate:

1. **Order lands.** Stripe takes payment; the webhook saves the order and emails you “New order #...”. (Until Phase 2, you can also just read it in the Stripe Dashboard.)
2. **Find the file.** Open the order; each line shows the photo + chosen frame/size. Use `source_path` to grab the full-res original from the drive.
3. **Place the print order.** On your supplier (e.g. Prodigy), upload/select the image, choose the matching frame + size + **acrylic** glazing, and enter the **customer’s** address as the shipping address (white-label / blind ship).
4. **Record it.** Mark the order `ordered` (note supplier + their order #).
5. **Ship + notify.** When the supplier despatches, add the tracking number and mark `shipped` — the buyer gets an automatic “on its way” email.
6. **Edge cases.** Breakage/returns: acrylic minimises this; keep a simple policy (replace damaged-in-transit on photo proof). If you ever self-ship, use proper corner-protected art boxes.

Time per order: ~5 minutes once you’ve done a couple. That’s very manageable at launch volumes, and Phase 3 removes even that.

Money, tax & the practical bits

- **Stripe fees (AU, approx — confirm current):** ~1.7% + A\$0.30 for domestic cards, ~3.5% for international. Factor into margin, especially the A3 tier. No monthly fee.
- **GST/ABN:** get an ABN; GST registration is only required at A\$75k turnover (optional below that). Stripe can add tax if you register later. **Confirm specifics with your accountant.**
- **Shipping pricing:** simplest is free shipping baked into the price (clean UX), or a flat rate per order. Stripe Checkout can collect the address and add a shipping line. Acrylic + small/medium sizes keep freight predictable.
- **Refund/returns policy:** made-to-order prints are generally non-returnable except for damage/defects — state this clearly; honour transit damage on proof. A short policy page builds trust.
- **Emails:** Resend (free tier ~3k/mo) for confirmation + shipped notifications; verify your domain so they come from `sam@samduckworth.com`.
- **Test before launch:** Stripe test mode end-to-end, a real \$1 live test, then flip `shop_public`.

Cost & effort summary

Item	Cost	When
Stripe	~1.7%+30c per sale, no monthly	Phase 1
Supabase / Vercel	\$0 (current plans cover this)	existing
Resend (email)	\$0 free tier	Phase 1
Supplier (print + frame)	per-order (see comparison)	per sale
Dev — Phase 1 MVP	~1–2 days	now
Dev — Phase 2 admin orders	~1 day	soon after
Dev — Phase 3 auto-dropship	~1–2 days	when volume warrants

Decisions to lock in (checklist)

- ☐ Launch supplier + account opened, real A3/A2/A1 framed cost in hand
- ☐ Frame finishes to offer (oak / black / white?) and glazing = acrylic for posted items
- ☐ Retail prices confirmed at ~2.5–3× landed cost (re-check A1)
- ☐ Free shipping baked in, or flat-rate?
- ☐ Stripe account + ABN; refund/shipping policy text
- ☐ Green-light Phase 1 build (cart + Stripe + orders + emails), then flip the shop live

Bottom line: you're close. The shop looks finished; it just can't take money yet. Phase 1 (~1–2 days) gets you selling with manual fulfilment on your own stack at the lowest fees, and nothing about it has to be rebuilt to automate later. Want me to start on the Phase 1 build (the migration, the two Vercel functions, and the real cart) whenever you've picked a supplier?